

Lab 4: VibeCoding - Chat History

Siddanth Anil · SRN PES1UG24AM274 · Section E · Repo: SETAPESU26/35_qbert (Q*bert)

LLM used: Claude (Opus 5.5) in claude.ai, working agentically (it cloned the repo, edited game.py, ran the game headless and recorded the videos). Only the first prompt was typed by me; Claude then split the work into one sub-task per README task, with one commit each. Below: each step, what Claude did, and the exact diff committed. The full live chat is at the [claude.ai share link](#).

Setup

My prompt: https://github.com/SETAPESU26/35_qbert - complete the deliverables (lab handout attached).

Claude: Cloned the repo, read README.md and game.py. Identified one deliberate bug (Task 1) and three stub functions (Tasks 2-4). Ran the original game headless with scripted key presses and recorded a 10 s 'before' video: the pyramid is visibly skewed (odd rows shifted half a cube right).

Task 1 - Fix the pyramid projection bug

Sub-task (set by Claude): Fix the skewed pyramid. Look at cube_center: should the row offset use integer or true division?

Claude: Row r has r+1 cubes, so to centre it the row must shift left by r/2 cube widths. row // 2 floors that value, so every odd row is off by half a cube, breaking symmetry. Changed to row / 2. Verified by rendering a frame: rows are now centred.

Commit 588903c

```
diff --git a/Lab-4/game.py b/Lab-4/game.py
index 38f8f46..9c8795b 100644
--- a/Lab-4/game.py
+++ b/Lab-4/game.py
@@ -26,7 +26,8 @@ def bonus_life_threshold():

def cube_center(row, col):
-    return pygame.Vector2(WIDTH / 2 + (col - row // 2) * CUBE_W, 90 + row * CUBE_H)
+    # Each row is shifted left by half a cube per row; true division keeps odd rows centred.
+    return pygame.Vector2(WIDTH / 2 + (col - row / 2) * CUBE_W, 90 + row * CUBE_H)

def neighbors(row, col):
```

Task 2 - Implement cube_palette(level)

Sub-task (set by Claude): Implement cube_palette so each level gets its own 3-colour palette (TARGET + 1 stages).

Claude: Returns one of four hand-picked palettes, cycling with (level - 1) % 4; level 1 keeps the original colours. Guarded so a palette of the wrong length falls back to DEFAULT_PALETTE (returns None). Verified levels 2 and 3 render magenta/yellow and grey/orange.

Commit 5484065

```
diff --git a/Lab-4/game.py b/Lab-4/game.py
index 9c8795b..756909c 100644
--- a/Lab-4/game.py
+++ b/Lab-4/game.py
@@ -12,7 +12,14 @@ KEY_HOPS = {pygame.K_LEFT: (-1, -1), pygame.K_UP: (-1, 0), pygame.K_DOWN: (1, 0)}

def cube_palette(level):
    """Return a list of TARGET + 1 (r, g, b) colours for the cube stages, or None for the default."""
-    pass
+    palettes = [
+        DEFAULT_PALETTE, # level 1: blue -> brown -> green
+        [(200, 80, 160), (240, 200, 60), (60, 200, 220)], # level 2: magenta -> yellow -> cyan
+        [(110, 110, 120), (230, 120, 40), (250, 250, 250)], # level 3: grey -> orange -> white
+        [(60, 140, 70), (150, 90, 200), (240, 90, 90)], # level 4: green -> purple -> red
+    ]
+    palette = palettes[(level - 1) % len(palettes)] # cycle once levels run past the list
+    return palette if len(palette) == TARGET + 1 else None

def on_cube_completed(cell):
```

Task 3 - Implement on_cube_completed(cell)

Sub-task (set by Claude): Implement on_cube_completed: give the completed cube a brief flash.

Claude: The hook gets no reference to the Game, so it records the flash in a module-level FLASHES dict (cell -> seconds left). Game.update counts timers down by dt, draw_cube blends the top face toward white proportional to the remaining time (fade-out over 0.4 s), and reset() clears stale flashes. Only fires on the hop that finishes a cube, since paint() calls it only when stage == TARGET.

Commit 3004e4c

```
diff --git a/Lab-4/game.py b/Lab-4/game.py
index 756909c..d049960 100644
--- a/Lab-4/game.py
+++ b/Lab-4/game.py
@@ -8,6 +8,8 @@ HOP_TIME, HOP_HEIGHT = 0.28, 26
CELLS = {(r, c) for r in range(ROWS) for c in range(r + 1)}
DEFAULT_PALETTE = [(90, 160, 220), (190, 120, 70), (100, 210, 140)]
KEY_HOPS = {pygame.K_LEFT: (-1, -1), pygame.K_UP: (-1, 0), pygame.K_DOWN: (1, 0), pygame.K_RIGHT: (1, 1)}
+FLASH_TIME = 0.4 # seconds a freshly completed cube flashes white
+FLASHES = {} # cell -> seconds of flash remaining

def cube_palette(level):
@@ -24,7 +26,7 @@ def cube_palette(level):

def on_cube_completed(cell):
    """Called when a cube first reaches its target colour; add a flash, sound, or bonus here."""
-    pass
+    FLASHES[cell] = FLASH_TIME # Game.update counts this down, draw_cube renders it

def bonus_life_threshold():
@@ -95,6 +97,7 @@ class Game:
    self.level, self.score, self.lives = 1, 0, 3
    self.bonus_awarded = 0
    self.stages = {cell: 0 for cell in CELLS}
+    FLASHES.clear()
    self.state = "play"
    self.respawn()

@@ -135,6 +138,10 @@ class Game:
    self.respawn()

    def update(self, dt):
+        for cell in list(FLASHES):
+            FLASHES[cell] -= dt
+            if FLASHES[cell] <= 0:
+                del FLASHES[cell]
        if self.state != "play":
            return
        landed = self.player.update(dt)
@@ -175,6 +182,9 @@ class Game:
    cx, cy = cube_center(*cell)
    top = [(cx, cy - CUBE_H / 2), (cx, cy + CUBE_H / 2), (cx, cy + CUBE_H / 2), (cx - CUBE_W / 2, cy)]
    color = colors[self.stages[cell]]
+    if cell in FLASHES: # blend toward white, fading out over FLASH_TIME
+        k = FLASHES[cell] / FLASH_TIME
+        color = tuple(int(c + (255 - c) * k) for c in color)
    left = [top[3], top[2], (cx, cy + CUBE_H / 2 + SIDE), (cx - CUBE_W / 2, cy + SIDE)]
    right = [top[1], top[2], (cx, cy + CUBE_H / 2 + SIDE), (cx + CUBE_W / 2, cy + SIDE)]
    pygame.draw.polygon(screen, shade(DEFAULT_PALETTE[0], 0.45), left)
```

Task 4 - Implement bonus_life_threshold()

Sub-task (set by Claude): Implement bonus_life_threshold to award a life every 1000 points.

Claude: Returns 1000. The existing bookkeeping (bonus_awarded = score // threshold) grants exactly one life per new multiple and persists across levels (reset(full=False) keeps it). Verified in the after video: Lives goes 2 -> 3 as the score crosses 1000.

Commit 6aa5467

```
diff --git a/Lab-4/game.py b/Lab-4/game.py
index d049960..440861f 100644
--- a/Lab-4/game.py
+++ b/Lab-4/game.py
@@ -31,7 +31,7 @@ def on_cube_completed(cell):
def bonus_life_threshold():
```

```
        """Return a score value at which the player earns an extra life, or None to disable bonus lives."""
-    pass
+    return 1000 # one extra life at 1000, 2000, 3000, ...

def cube_center(row, col):
```

After video

Sub-task (set by Claude): Record the after video showing the fix and every feature.

Claude: Recorded 10 s of the fixed build: symmetric pyramid, flash on completed cubes, bonus life at 1000, then levels 2 and 3 to show the new palettes. To fit everything in 10 s the demo script sets the score to 960 and skips to the next level; all rendering and game logic are the real game.py code.